Relational Database Systems –

COSC265

Lectures 27 & 28 – Query Optimization

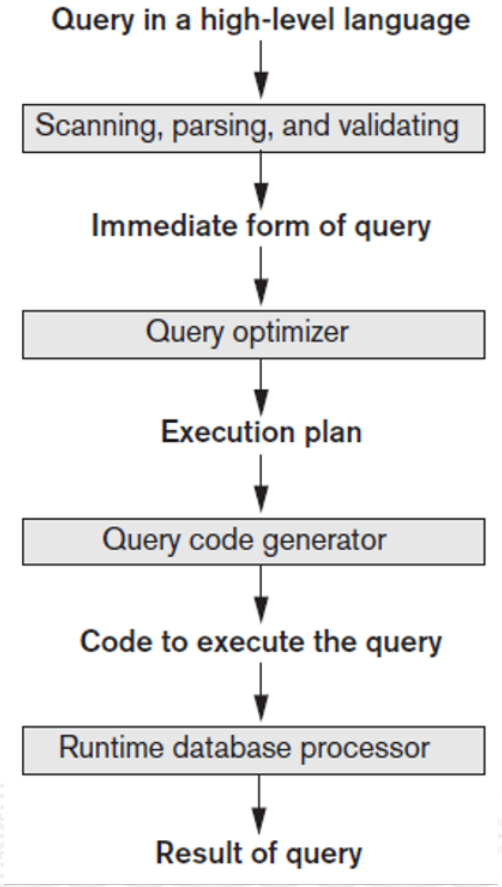


Miguel Morales

UC UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

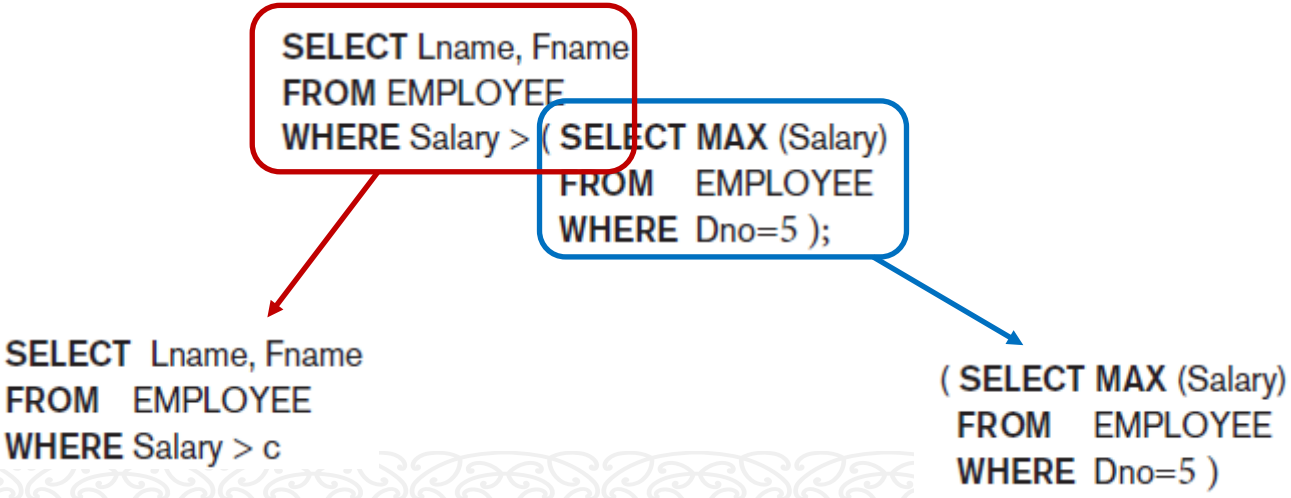
Query Processing

- DBMS steps to process a query
  1. Scanning to identify query tokens
  2. Parsing to check the query syntax
  3. Validating all attribute and relation names
  4. Creating a query tree (or query graph)
  5. Developing an execution strategy or query plan
  6. Optimizing the query
  7. Generating the code and executing the query

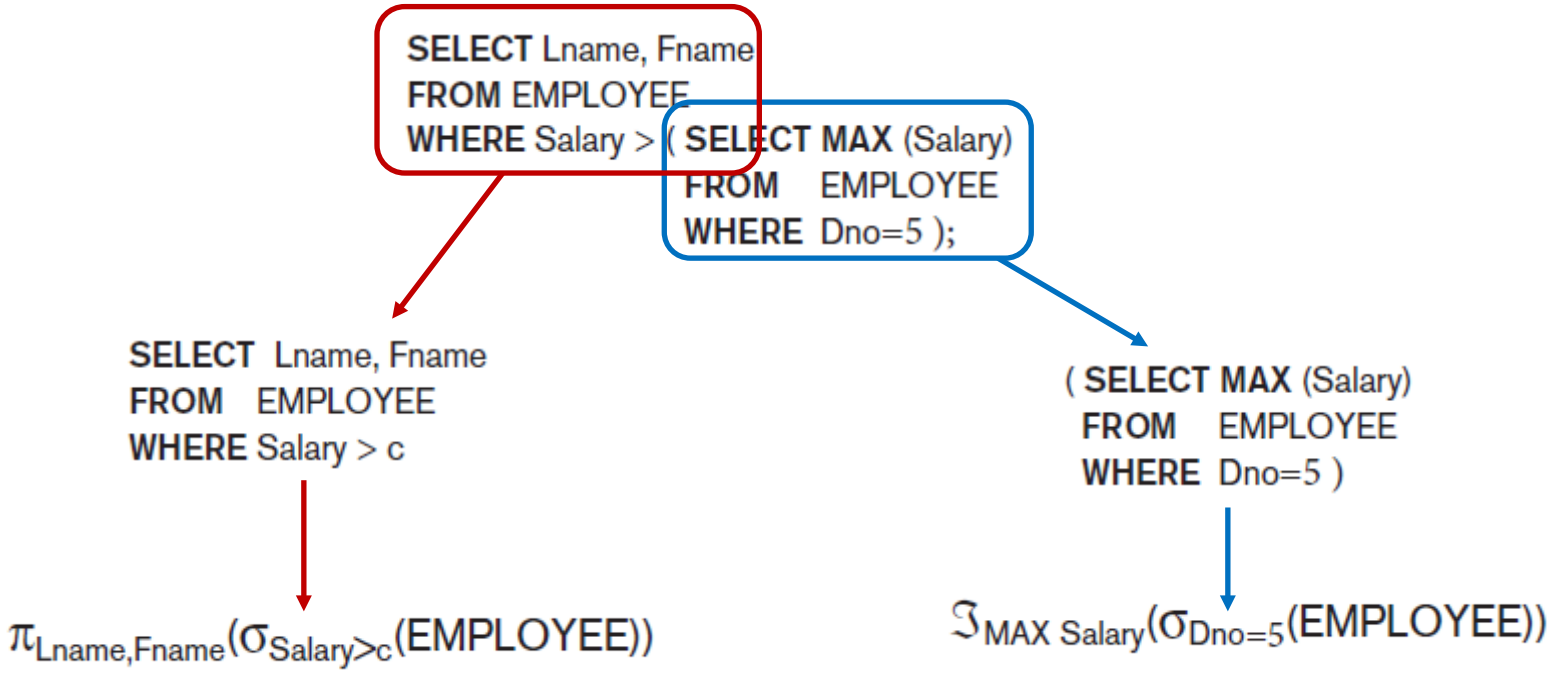


Query Processing

- **Query Block**
  - The **basic unit** that can be **translated into algebraic operators** and optimized
  - A single SELECT-FROM-WHERE expression, potentially including GROUP BY and HAVING
  - Nested queries within a query are identified as separate query blocks



Query Processing



– The Query Optimizer then chooses an **execution plan for each query block**

SELECTION Operation

- **SELECTION** operation
  - It is a search operation to locate records in a disk file that satisfy a certain condition
- **Search methods** for selection
  - S1: Linear search (brute force)
  - S2: Binary search
  - S3: Using a primary index
  - S4: Using a hash key
  - S5: Using a clustering index
  - S6: Using a secondary index

How does SELECTION affect the efficiency of a query?

- **Selectivity** of a search condition
  - It is **the ratio of the number of tuples that satisfy the condition** to the total number of tuples in the relation (between 0 and 1)

JOIN Operations

- **JOIN** operations
  - One of the most time-consuming operations in query processing

- Methods for implementing joins
  - J1: Nested-loop join
  - J2: Index-based nested-loop join
  - J3: Sort-merge join
  - J4: Partition-hash join

What should we avoid when choosing the type of JOIN?

PROJECT Operation

UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- **PROJECT** operation
  - After projecting only the columns in the list of attributes, any duplicates are removed by treating the result strictly as a set of tuples

- Default for SQL queries
  - No elimination of duplicates from the query result
  - Duplicates are removed only if the keyword **DISTINCT** is included

Why is PROJECT implemented in SQL differently
to its Relational Algebra definition?

Aggregate Operations

UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- **Aggregate operators**
  - These can be computed by a table scan or using an appropriate index
- **AVERAGE** or **SUM**
  - An index can be used if it is a dense index
  - Computation applied to the values in the index
- **COUNT**
  - The number of records can be computed from the index

Would it make sense to use an index to optimize **MIN**/**MAX**?

Query Optimization

UC
UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- **Query tree**
  - A **tree data structure** that corresponds to a **relational algebra expression**
  - Relations → leaf nodes
  - Operations → internal nodes

- An execution of the query tree consists of:
  - Executing an internal node operation whenever its operands are available
  - Then replacing that internal node with the resulting relation

- **Canonical tree**
  - A standard tree that corresponds to an SQL query without optimization

Query Optimization

- Find all the Dallas Cowboys players.



SQL:

```
SELECT *
FROM player, player_team, team
WHERE player.player_id = player_team.player_id
   AND team.team_id = player_team.team_id
   AND team_nickname LIKE 'Cowboys';
```

Relational Algebra:

$$\pi_{*}\left(\sigma_{\substack{player.player_id=player_team.player_id \wedge \\ team.team_id=player_team.team_id \wedge \\ team_nickname='Cowboys'}}(player \times player_team \times team)\right)$$

COSC265 QOptimization.sql

Query Optimization

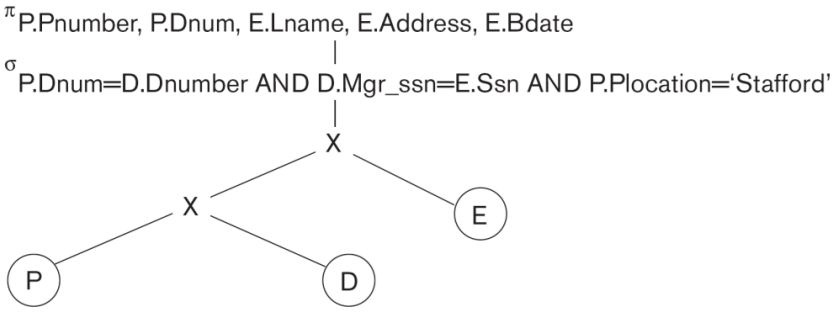
- For every project located in ‘Stafford’, retrieve the **project number**, the controlling **department number** and the department manager’s **last name**, **address** and **birthdate**.

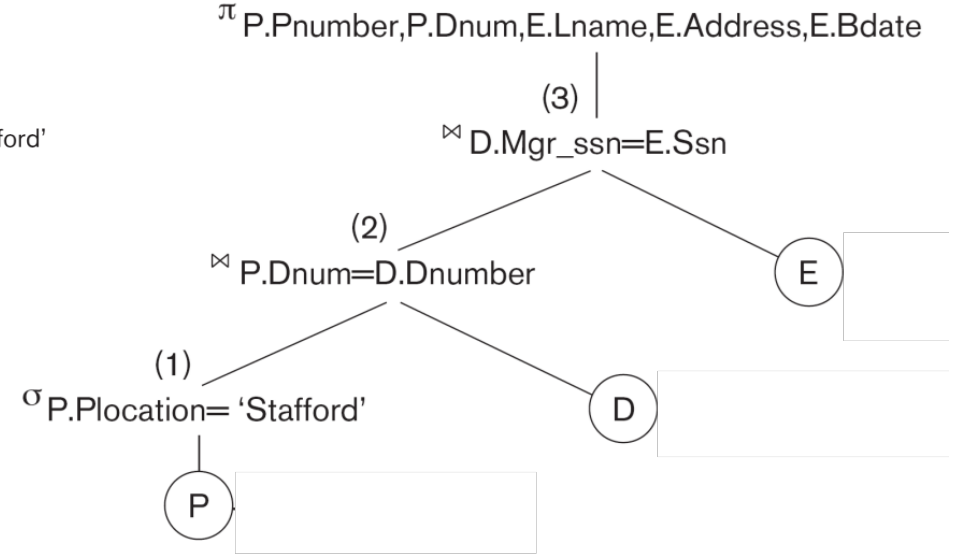
SQL:

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM Project P, Department D, Employee E
WHERE P.Dnum=D.Dnumber AND D.Mgr_ssn=E.Ssn AND P.Plocation=‘Stafford’;
```

$$\pi_{Pnumber,\ Dnum,\ Lname,\ Address,\ Bdate}\left(\sigma_{\substack{P.Dnum=D.Dnumber\ \wedge \\ D.Mgr_ssn=E.Ssn\ \wedge \\ P.Plocation='Stafford'}}(Project \times Department \times Employee)\right)$$

Query Optimization





Which one is the canonical tree?

Which one is most efficient? Why?

Query Optimization

UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

1. **Parse** → Create an internal representation of the query
2. **Optimize** → Apply heuristics to improve the representation
3. **Plan** → Select operations and access paths for execution

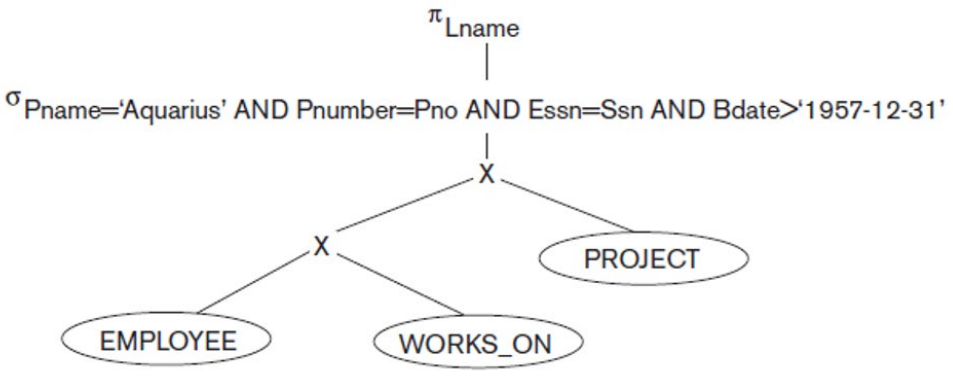
- ***Reduce*** intermediate results
  - Apply operations that **reduce data size as early as possible**
- Select and project early
  - Apply SELECT (σ) and PROJECT (π) before JOIN (⋈) whenever possible
- Choose the best [available] plan
  - Compare alternative execution plans and select the one with the lowest estimated cost

Cost of *what*?

Query Optimization

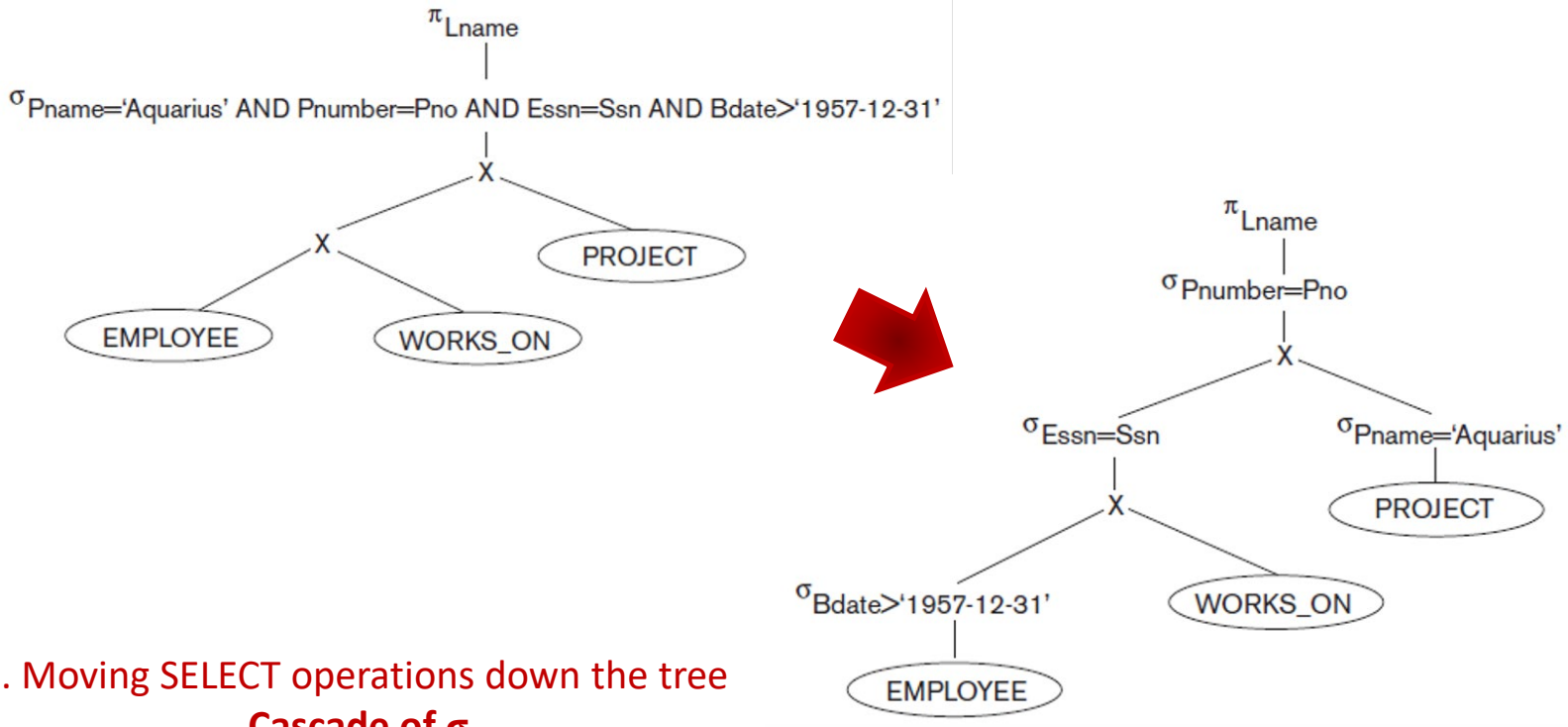
- One query → Many equivalent query trees
- The DBMS's goal [ours too] is to transform the initial query tree into a more efficient execution form
- Example:

```
SELECT LNAME
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE      PNAME = 'Aquarius' AND
           PNUMBER=PNO AND
           ESSN=SSN AND
           BDATE > '1957-12-31';
```



0. Canonical tree – no optimizations (yet)

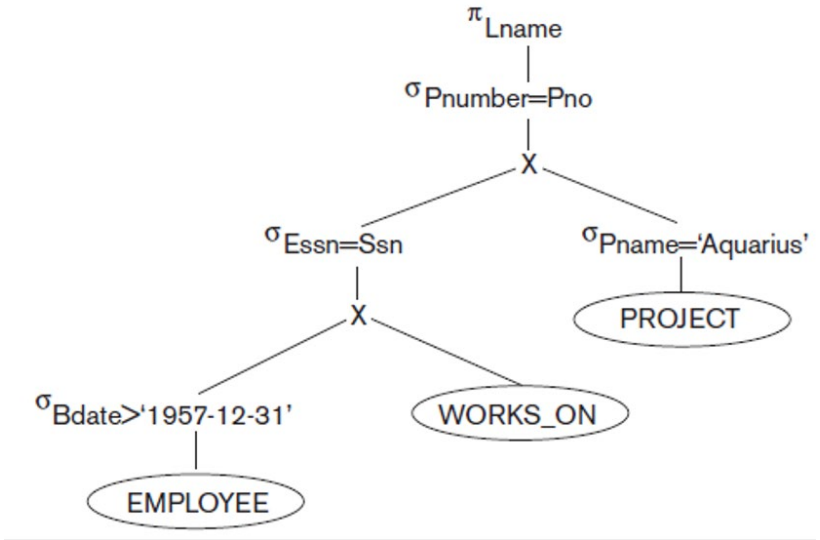
Query Optimization

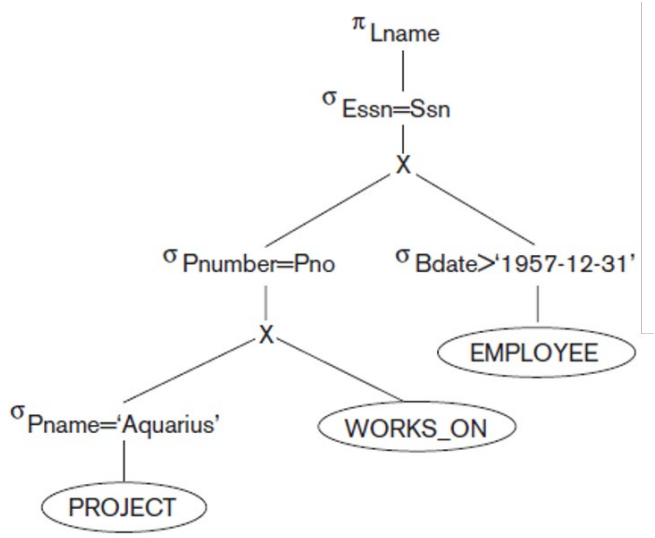


1. Moving SELECT operations down the tree

Cascade of σ

Query Optimization

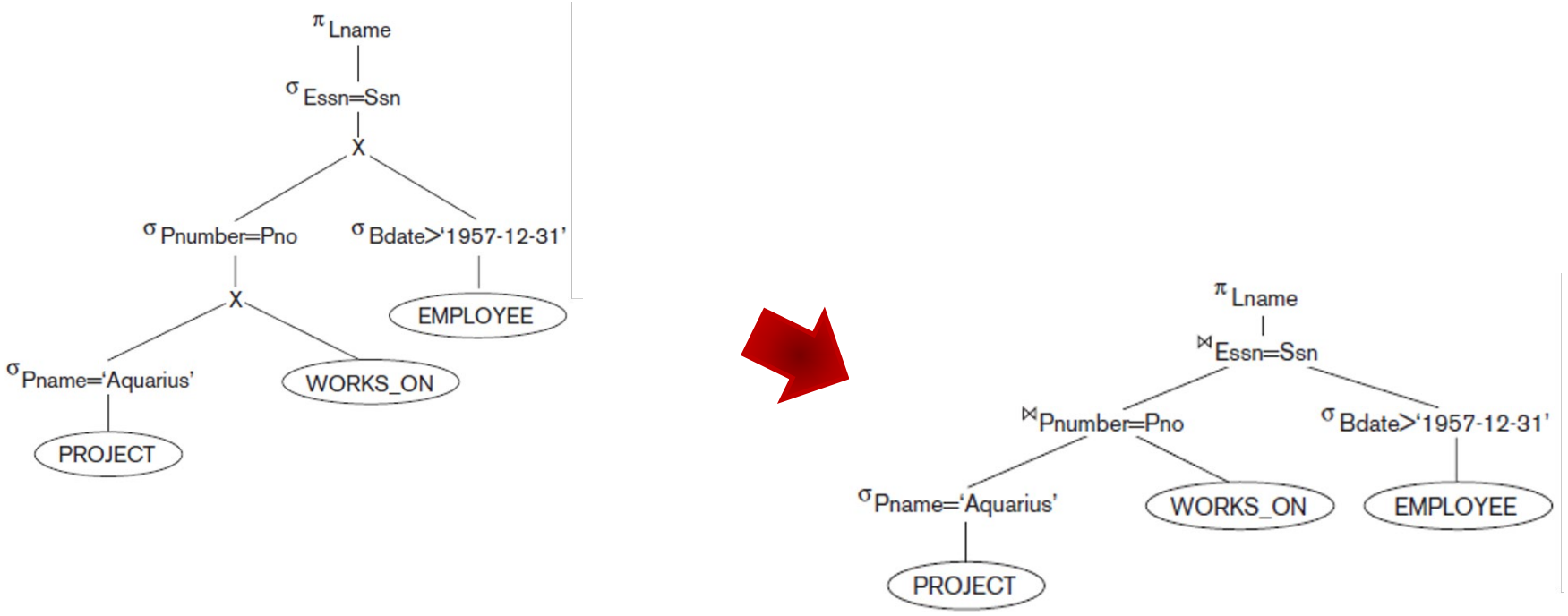




2. Applying the *more restrictive* SELECT operation first

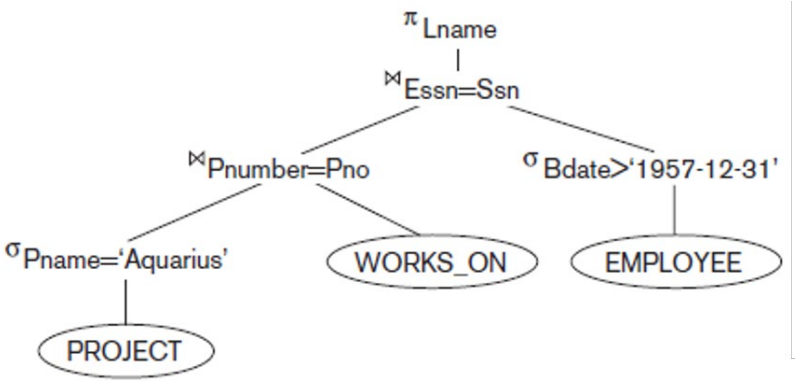
Commutativity of σ

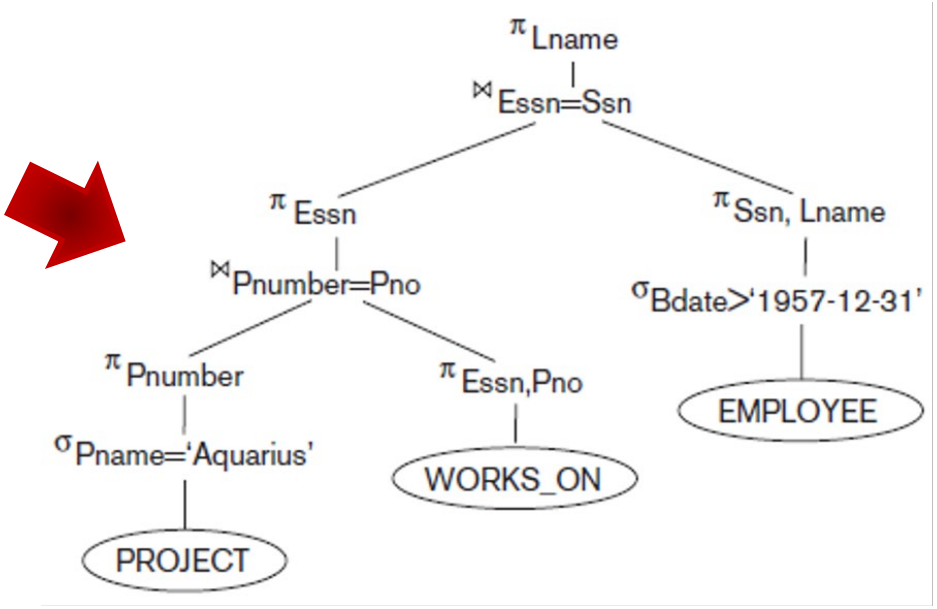
Query Optimization



3. Replacing JOINs with more efficient and appropriate ones

Query Optimization





4. Moving PROJECT operations down the tree

Cascade of π

Types of Evaluation

UC
UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- **Materialized** evaluation
  - The result of an operation is stored as a temporary relation
- **Pipelined** evaluation
  - The operation results are forwarded directly to the next operation in the query sequence
- Why materialize?
  - The stored results may be maintained **temporarily or persistently**
  - Reading precomputed results can be much faster than recomputing them
- **Materialized views**
  - A materialized view stores the results of a predefined query [which was already optimized]
  - These are an optimization technique that can avoid repeating expensive computations

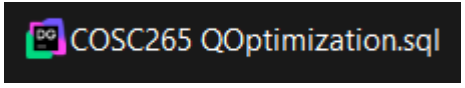
Query Execution

- **Rule-based** query optimization
  - The optimizer chooses execution plans based on heuristically ranked operations

- **Cost-based** query optimization
  - The optimizer examines alternative access paths and operator algorithms and chooses the execution plan with the lowest estimated cost (e.g., I/O, CPU, and memory needed)

```
EXPLAIN PLAN FOR
SELECT * FROM player WHERE player_surname LIKE 'Edwards' OR player_surname LIKE
'Simmons';

-- CHECK THE RESULTS
SELECT plan_id, timestamp, cpu_cost
FROM PLAN_TABLE;
```



Cost Components

- Cost components for query execution
  - **Access** cost to secondary storage
  - **Storage** cost
  - **Computation** cost
  - **Memory usage** cost
  - **Communication** cost

Different DBMSs may focus on different cost components

- Most of this data can be obtained from the DBMS **catalogs**

Catalogs

- **Catalog** information used in cost functions:
  - Information about the **size of a file**
  - Number of **records**
  - Record size
  - Number of **blocks** and **blocking factor**
  - Information about **indexes** and indexing attributes
  - Number of **levels** of each multilevel index
  - Number of first-level index blocks
  - Number of **distinct values** of an attribute
  - **Selectivity** of an attribute

Catalogs

- Data catalogs are **descriptions of the database** in terms of their objects
  - The metadata of the database: *"data about data"*
- Catalogs are stored as a set of **read-only tables and views**
  - For each **table**: table name, rows, file name, and size
  - For each **attribute**: attribute name and type
  - For each **index**: index name, indexing attribute(s), type, file name, and size
  - For each **view**: view name and definition (query)
  - For each **role**: role name, privileges, authorization info
- **User-accessible views:**
  - **USER** views (objects owned by user)
  - **ALL** views (objects to which user has privileges)
  - **DBA** views (all objects)

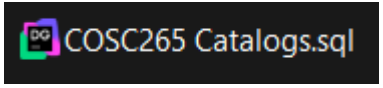
Catalogs

- Examples:
  - **ALL_CATALOG** (owner, table_name, table_type)
  - **ALL_OBJECTS** (object_name, object_id, object_type, created, . . .)
  - **USER_CATALOG** (table_name, table_type)
  - **USER_TABLES** (table_name, tablespace name, . . . ,pct_free, initial_extent, num_rows, blocks,empty_blocks, avg_row_length)
  - **USER_TAB_COLUMNS** (table_name, column_name, data_type, . . . , nullable, . . . , num_distinct, low_value, high_value, density, num_nulls, . . . , last_analyzed, global_stats, user_stats, avg_col_length)
  - **USER_CONSTRAINTS** (owner, table_name, constraint_name, constraint_type, search_condition, . . .)
  - **USER_VIEWS** (view_name, text_length, text, . . .)
  - **USER_INDEXES** (index_name, index_type, . . . , blevel, leaf blocks, distinct_keys, avg_data_blocks_per_key)

SELECT *

FROM ALL_CATALOG

WHERE OWNER = ‘Miguel’;



ANALYZE TABLE aircraft **COMPUTE STATISTICS**;